

Feedback Studio

Prototype architecture, LangGraph building blocks, Jev decision routing, agent roles, and operating guardrails.

01 · Product workflow

Customers scan a QR code, rate food taste, waiting time, ambience, and staff service, then optionally write a comment and attach a photo. The owner dashboard separates possible emergencies from service improvements and shows an analysis of all feedback.

CUSTOMER	INTAKE	ROUTING	OWNER
QR feedback form ratings · comment · optional photo	FastAPI validate · persist · identify submission	LangGraph + Jev emergency · improvement · routine	Phone alert for possible emergency dashboard for every response

Possible emergency reports include a pest or foreign object in food, suspected contamination, allergen exposure, illness reportedly linked to food, or another plausible immediate safety risk. Poor taste, long waits, ambience, and staff-service concerns go to the improvement list without interrupting the owner by phone.

02 · Architecture and tech stack

Layer	Prototype choice	Responsibility
Customer form	Google Form / built-in QR form	Ratings, free text, optional image
Ingestion	Apps Script + FastAPI	Submit event, validation, webhook auth
Orchestration	LangGraph StateGraph	Typed shared state, conditional routes, node boundaries
Decision model	Ollama Jev / TypeSafe	Fast typed urgency and route decision
Vision review	Ollama vision model (optional)	Classify photo relevance; never gate text safety alerts
Persistence	SQLite	Feedback, workflow result, notification outbox
Owner UI	FastAPI + Jinja + CSS	Read-only dashboard, emergency and improvement views
Phone alert	Twilio SMS adapter (optional)	Send only possible emergency notifications

Google Apps Script runs outside the local machine. It cannot reach localhost; use a temporary HTTPS tunnel for a connected local demo or deploy the API to a secure HTTPS host. Google Forms file-upload questions require respondents to sign in to a Google Account.

03 · LangGraph building blocks

LangGraph keeps the workflow explicit and reviewable. The prototype uses bounded nodes and conditional edges, not open-ended model-driven tool loops.

Building block	Use in Chai House
State	One typed FeedbackState carries the feedback ID, ratings, comment, image path, urgency, category, summary, and notification status.
Node	A named function performs one bounded job: Jev triage, image review, emergency alert preparation, improvement routing, persistence, or analysis.
Conditional edge	Routes emergency text to the emergency agent; sends reports with a photo through optional image review first; sends non-emergency feedback to improvement or routine handling.
Persistence	SQLite records both the response and notification outbox so processing status and alert outcome are visible.
Fallback	If Ollama is unavailable, deterministic hazard terms and ratings classify the report; failures are saved for owner review.

Graph outline

```
START -> Jev triage -> { emergency + photo: image review -> emergency agent; emergency + no photo: emergency agent; improvement: improvement agent; routine: routine node } -> persist result -> END
```

Dashboard analysis uses a separate LangGraph StateGraph and a bounded analyst node. SQL computes counts and average ratings first; the narrative model receives only those precomputed metrics and must not invent numbers.

04 · Three agent roles

- 01

Emergency review
Uses Jev triage plus optional image relevance review. Produces a concise owner alert. Credible text reports trigger the alert even when no photo is attached.
- 02

Improvement feedback
Groups taste, wait, staff, and ambience feedback and gives a practical review action. Does not send owner phone alerts.
- 03

Dashboard analysis
Summarizes supplied metrics, supported positive themes, recurring improvement areas, and next actions. Counts are calculated by code.

05 · Guardrails and safety policy

The model classifies customer reports; it does not verify incidents or make food-safety determinations. Alert rules belong to application code and should be reviewed with the cafe owner before live operation.

- **No photo gate** — A plausible immediate safety report triggers an owner alert whether or not a photo is supplied. Image processing can fail or be inconclusive without blocking the alert.
- **Images are evidence to review** — A photo may support or conflict with a report, but cannot prove when or where it was taken. A photo alone without a relevant report does not create an emergency.
- **Unverified wording** — Use 'customer reported' and 'possible'. Never phrase an allegation as a confirmed event.
- **Hard policy override** — Explicit hazard terms are protected by a deterministic rule; Jev cannot downgrade them. Model uncertainty is saved for owner review.
- **Separation of outcomes** — Possible immediate risk -> phone alert; service/quality concerns -> dashboard improvement queue; all reports -> analysis.
- **No invented analytics** — SQL calculates counts and averages. The analysis agent may describe only supplied metrics and should say 'not enough data' for weak patterns.
- **Data minimization** — Avoid collecting names, phone numbers, or email unless a later operational need is approved. Keep photos optional and limit their size and type.
- **Notification deduplication** — Use a unique feedback ID and one outbox entry per report to reduce duplicate SMS alerts.
- **Human review** — The dashboard labels reports as customer-reported. The owner remains responsible for deciding what operational response to take.

06 · Jev decision contract

Jev is the routing decision step and the main efficiency factor: one request can answer typed questions such as urgency and route. It is not the image model, the notification provider, or the source of computed dashboard statistics.

Input: feedback text and four 1-5 ratings. Output urgency: emergency | improvement | routine. The workflow validates the label, applies hard safety overrides, then follows a known LangGraph edge. The local Jev-style endpoint requires Ollama 0.35+ and a decision model such as nimble. Older versions or unavailable models use a deterministic fallback.

Suggested production review

Before relying on live alerts, review examples with the owner, especially multilingual and misspelled reports; measure missed emergencies and false alarms; confirm who receives SMS and at what hours; and secure the dashboard behind owner authentication before public deployment.

07 · Prototype boundaries

The prototype includes a mobile form, SQLite persistence, a LangGraph workflow, Jev and Ollama adapters, optional Twilio SMS, dashboard filters, a generated feedback QR, and this architecture PDF. Live Jev, vision, Google Forms, and SMS behavior require local model availability and credentials/configuration. Without those settings, the app remains runnable using deterministic fallback and demo-logged alerts.

Local demo URL: `http://127.0.0.1:8000` · Health: `/health` · QR: `/static/qr-feedback.png` · API: `/api/dashboard`